

CanSat Autonomous Navigation: Algorithm Evaluation Report

Scope: Kalman filter + PID simulation (`kalman.py` , `proportional.py` , `integral.py` , `differential.py` , `pid.py`) evaluated for a parafoil (paraglider) CanSat steered by servo-actuated brake lines, released from a drone at 1000 m.

1. Executive summary

The current work is a sound **proof of concept** for the sense → filter → control pipeline. The Kalman filter, the PID structure with anti-windup and saturation, and the closed-loop simulation are all useful foundations.

It is **not yet representative of a real parafoil**, for one central reason: the simulation treats the vehicle as a point that can move in any direction at a commanded velocity. A parafoil cannot do this. It flies forward at a roughly fixed airspeed, loses altitude continuously, can only **turn** (by pulling a brake line), and is pushed around by wind that may be comparable to its own airspeed. The controller therefore needs to control **heading (turn rate)**, not latitude and longitude directly.

Key recommendations

1. Replace per-axis position control with a **guidance layer (bearing to target) + heading PID** that commands the servos.
2. Work in **local metres (ENU)**, not degrees.
3. Upgrade the plant model: forward airspeed, sink rate, turn dynamics, servo lag, wind.
4. Estimate **heading and wind**, and manage **altitude/energy** so the vehicle arrives at the target and can land there.
5. Validate with Monte Carlo simulation, hardware-in-the-loop, and staged drop tests.

2. Mission context and assumptions

Item	Assumed value	Note
Release altitude	1000 m	From a drone

Item	Assumed value	Note
Sink rate	3–6 m/s	Small parafoil, depends on loading
Airspeed	6–10 m/s	
Glide ratio (L/D)	about 2–3	Small ram-air/round-glide canopies
Descent time	about 170–330 s	1000 m divided by sink rate
Still-air glide range	about 2–3 km	altitude × L/D
Wind aloft	3–10 m/s	Can equal or exceed airspeed
GPS update rate	1–10 Hz	Typical modules
IMU / barometer rate	50–200 Hz	

These are planning figures, not measurements. Replace them with data from your canopy and sensors.

Feasibility check: with 1000 m of height and $L/D = 3$, the vehicle can cover about 3 km over the ground in still air. A 5 m/s wind over a 250 s descent displaces it about 1.25 km. Therefore **the release point selection (upwind of the target, within the reachable footprint) matters as much as the controller**.

3. Current algorithm summary

1. A noisy position measurement is generated at 100 Hz.
2. One 1D Kalman filter per axis (state = position, velocity; constant-velocity model) smooths it.
3. A controller computes $u = K_p \cdot e + K_i \cdot \int e + K_d \cdot de/dt$ on each axis, where $e = \text{target} - \text{estimate}$, and the output is saturated to u_{max} .
4. The plant integrates u (plus a constant wind drift) directly into position.

4. Strengths

#	Strength	Why it matters
S1	Modular pipeline (sensor → filter → controller → plant)	Each block can be improved or swapped independently.
S2	Closed-loop simulation rather than open-loop post-processing	Reveals saturation, lag and stability issues, as seen in the longitude run.

#	Strength	Why it matters
S3	Kalman velocity estimate used for the D term	Avoids differentiating noisy data directly.
S4	Anti-windup and output saturation	Models actuator limits and prevents integrator blow-up.
S5	P/I/D term breakdown plots	Makes tuning and debugging transparent.
S6	Wind disturbance included	Demonstrates the role of the I term (steady-state offset removal).
S7	Parameterized gains, structure ready for auto-tuning	Optimizers (e.g. differential evolution) can be applied directly.
S8	Anti-windup clamp tied to <code>u_max</code>	Prevents the integral term alone from exceeding the actuator limit.

5. Weaknesses and how to overcome them

Ranked by impact on a real flight.

W1. Wrong plant: holonomic point mass instead of a parafoil (critical)

- **Problem:** The simulation lets the vehicle move toward the target at any commanded velocity. A parafoil moves along its heading at near-constant airspeed, turns at a rate set by brake deflection, and cannot stop or reverse. A P controller on position would command motion the canopy cannot perform.
- **Fix:** Use a kinematic model such as:
 - $\dot{x} = V \cdot \cos(\psi) + w_E, \dot{y} = V \cdot \sin(\psi) + w_N, \dot{z} = -V_{\text{sink}}$
 - $\dot{\psi} = K_{\text{turn}} \cdot \delta$ (turn rate proportional to brake deflection δ), passed through a first-order servo/canopy lag
- Tune and validate the controller on this model first.

W2. Controlled variable is position, not heading (critical)

- **Problem:** Independent lat/lon PIDs have no meaning for a vehicle steered by turning.
- **Fix:** Add a guidance layer.
 - Compute bearing and distance from the estimated position to the target.
 - Heading error $e_\psi = \text{wrap}(\psi_{\text{desired}} - \psi_{\text{est}})$ to $\pm 180^\circ$.

- Heading PID outputs a normalized steering command (−1 to +1), mapped to left/right brake servo travel.

W3. Units in degrees; unrealistic noise (high)

- **Problem:** Noise with std 5 in degrees corresponds to hundreds of kilometres. Latitude and longitude degrees are not equal distances (longitude shrinks with $\cos(\text{latitude})$). Gains have no physical meaning.
- **Fix:** Convert to a local ENU frame in metres relative to the release or target point. Use realistic GPS noise (about 1.5–3 m horizontal) and a realistic barometer noise.

W4. Filter model and structure too simple (high)

- **Problem:** Two independent constant-velocity filters have no heading, no wind, no altitude coupling and no use of the IMU. The constant-velocity model lags during turns. Treating a heading state as a plain number also breaks at the $\pm 180^\circ$ wrap.
- **Fix:**
 - Use a multi-rate **EKF** (or error-state KF): predict with IMU at high rate, correct with GPS when a fix arrives, with barometer and magnetometer corrections.
 - State: position, velocity, heading, gyro bias, and optionally wind.
 - Handle angle wrapping in the innovation.

W5. Sampling rate assumption (high)

- **Problem:** The simulation assumes a 100 Hz position measurement. Typical GPS provides 1–10 Hz.
- **Fix:** Simulate sensors at their true rates. Run the filter prediction at IMU rate and apply GPS updates only when available. Run the control loop at 20–50 Hz.

W6. No wind handling (high)

- **Problem:** Wind is a constant offset in the simulation. Real wind changes with altitude, may exceed the canopy airspeed, and creates crab angle (heading differs from ground track).
- **Fix:**
 - Steer using **ground-track** bearing, or estimate wind from the difference between ground velocity and airspeed (a circling manoeuvre improves observability).
 - Use a wind-corrected guidance law (e.g. L1 or pure pursuit with crab compensation).
 - Choose the release point upwind of the target using a forecast wind profile.

W7. No altitude / energy management and no landing logic (high)

- **Problem:** The controller converges asymptotically on a point. A glider never reaches a point and stops; it arrives at ground level, possibly too high (overshoots) or too low (falls short).
- **Fix:**
 - Compute **reachable range** continuously: $\text{altitude} \times L/D$ versus distance to target, corrected for wind.
 - If too high, burn altitude with an S-turn or spiral loiter around the target; if too low, fly the most efficient straight line.
 - Use a capture radius, landing approach into the wind, and optional flare near the ground.

W8. No actuator model (medium-high)

- **Problem:** Servos have rate limits, backlash, deadband, travel limits and lag. Brake-line pulls can be asymmetric and nonlinear.
- **Fix:** Model rate limiting, deadband, travel saturation and first-order lag. Calibrate the steering-command $\rightarrow$ turn-rate map from test flights. Use output rate limiting in the controller.

W9. Integral and derivative behaviour not matched to the problem (medium)

- **Problem:** Integral action on heading can wind up during turns or saturation; a large K_d amplifies noise when heading comes from a noisy sensor.
- **Fix:** Use conditional integration (stop integrating when saturated), a small K_i , derivative on the measurement rather than the error, and a low-pass filter on D. Consider gain scheduling by airspeed or altitude.

W10. Simulation covers a single noisy run (medium)

- **Problem:** One random run says little about reliability. The simulation length is tied to the CSV (2000 samples = 20 s), unrelated to descent time.
- **Fix:** Simulate the full descent (200–300+ s). Run Monte Carlo trials over wind speed/direction, release point, noise, sensor dropouts and canopy parameters. Report landing-error statistics (mean, 95th percentile, miss rate).

W11. No state machine or failsafes (high for flight)

- **Problem:** No handling of GPS loss, bad heading, sensor failure, or the drone-release transient.
 - **Fix:** Implement phases: ground $\rightarrow$ carried by drone $\rightarrow$ release $\rightarrow$ canopy deployment/stabilization $\rightarrow$ guided descent $\rightarrow$ landing approach $\rightarrow$ landed. Add failsafes: on GPS loss, hold neutral or last good turn command and fly straight; on low battery or servo fault, go neutral; apply a geofence.
-

6. Release from the drone: specific considerations

- **Release transient:** expect tumbling or pendulum motion before the canopy is stable. Keep the controller neutral for several seconds after release or deployment and until heading and velocity estimates converge.
- **Initial conditions:** initialize the filter from the drone's last known position and heading, then correct with GPS.
- **Release-point planning:** use the reachable-footprint calculation (Section 2) to pick a release point upwind of the target with margin.
- **Regulation and safety:** dropping a payload from 1000 m generally needs airspace permission and a safe, cleared landing area. Check local aviation rules (drone altitude limits are often far lower than 1000 m) and event rules.

7. Validation plan

Stage	What to test	Pass criterion (example)
1. Software-in-the-loop	Full descent in the improved model, Monte Carlo (hundreds of runs)	Landing within a defined radius (e.g. 95% inside X m)
2. Sensor/estimator test	Replay real logged GPS/IMU data through the EKF	Estimate error within bounds; heading stable
3. Hardware-in-the-loop	Flight code on the real microcontroller with simulated sensors	Loop timing met; no overruns
4. Servo bench test	Step and ramp commands, measure lag and limits	Servo model matches measured response
5. Low-altitude drop tests	Release from low height, log everything	Controller turns correctly; no instability
6. Progressive altitude increase	Increase release height step by step	Behaviour matches simulation
7. Full mission drop	1000 m release	Meets landing accuracy goal

Use logs from each stage to refine the plant model and re-tune the gains (the auto-tuning approach discussed earlier works well here).

8. Prioritized roadmap

1. Convert to ENU metres and realistic sensor noise.
 2. Build the kinematic parafoil plant with wind and servo lag.
 3. Implement guidance (bearing) + heading PID.
 4. Upgrade to a multi-rate EKF with heading (and later wind).
 5. Add energy management (reachable-range check, loiter, landing approach).
 6. Add the flight state machine and failsafes.
 7. Run Monte Carlo, auto-tune the gains, then hardware-in-the-loop.
 8. Staged real drop tests; refine the model with flight data.
-

9. Conclusion

The existing code is a good starting platform and its architecture is right. The decisive gap is the physical model: a parafoil is a heading-controlled, wind-driven, descending vehicle, not a point you can push toward a coordinate. Moving to heading-based guidance in metres, adding wind, actuator and altitude handling, and validating through Monte Carlo and staged drop tests will turn the current simulation into a credible design for a 1000 m autonomous landing.